INTELLIGENT NOSQL DATABASES & CLOUD COMPUTING

PROJECT REPORT

Head-2-Code: A Real-Time Gamified Competitive Coding & Cloud-Powered Assessment Platform



LOVELY PROFESSIONAL UNIVERSITY

SCHOOL OF COMPUTERSCIENCEANDENGINEERING
LOVELY PROFESSIONAL UNIVERSITY

Submitted by:

Tusar Goswami

Registration No: 12315905

Programme and Section: B. Tech Computer Science and Engineering

Course Code: CSE-494

Under the Guidance of

Vikas Mangotra

Discipline of CSE/IT
Lovely School of Computer Science and Engineering
Lovely Professional University, Phagwara

TABLE OF CONTENTS

1. **Abstract** **3**
2. **Introduction** **4**
3. **Problem Statement** **6**
4. **Objectives** **7**
5. **System Specifications & Data Model Architecture** **8**
6. **Tools and Technologies** **11**
7. **Methodology & Implementation Pipeline** **13**
8. **System Architecture & Core Platform Modules** **16**
9. **MongoDB Database Design, Schema & CRUD Operations** **20**
10. **Real-Time WebSocket Engine, Online Judge & AI Integration** **24**
11. **Performance Evaluation, Benchmarking & Testing** **27**
12. **Results and Discussion** **31**
13. **Future Enhancements** **34**
14. **Conclusion** **35**
15. **References** **36**

1. ABSTRACT

The modern software engineering landscape places unprecedented emphasis on algorithmic problem solving, computational thinking, and rapid coding proficiency. While conventional online judges such as LeetCode, Codeforces, and HackerRank have established standardized repositories for data structures and algorithms (DSA) practice, they overwhelmingly suffer from pedagogical isolation, lack of real-time competitive peer interaction, and delayed or non-explanatory compilation verdicts. Novice and intermediate developers frequently face demotivation due to opaque error traces, the absence of guided hints, and solitary practice environments that fail to mirror the urgency of live competitive contests or technical interviews.

To overcome these fundamental limitations, this project presents Head-2-Code: an enterprise-grade, real-time gamified competitive coding platform and cloud-powered assessment ecosystem. Built natively on the MERN technology stack (MongoDB Atlas, Express.js, React.js 18 with Vite, and Node.js), Head-2-Code bridges the divide between deep asynchronous algorithmic problem solving and synchronized peer-to-peer competition.

At the core of the platform is a high-throughput WebSocket infrastructure powered by Socket.IO, establishing bidirectional, event-driven communication for 1v1 Friend Battle Arenas and live contest lobbies. The system achieves sub-300ms end-to-end event latency under concurrent active user loads, synchronizing live keystroke updates, rival testcase progress, and match results. In addition, an anti-cheat detection engine actively tracks browser tab switches and visibility events, applying automated time penalties and disqualification protocols to ensure competitive integrity.

Code compilation and verification are orchestrated via a distributed Judge0 sandbox execution engine supporting over 10 programming languages (including C++, Java, Python 3, and JavaScript). Submissions are evaluated against hidden test suites in parallel batches, yielding an average execution verdict response time of ~250ms. To address the guidance gap, Head-2-Code embeds Google's state-of-the-art Gemini Generative AI directly within the Monaco browser editor. The AI engine acts as a pedagogical tutor—performing AST-informed time and space complexity evaluations, identifying logic flaws, and supplying Socratic hints without exposing direct solutions.

The data persistence tier is driven by MongoDB Atlas with Mongoose ODM, utilizing compound indexes for rapid query resolution and structured document schemas for users, problems, submissions, friend arenas, contests, daily challenges, and badges. An in-memory Redis caching layer accelerates leaderboard aggregations and frequent read paths, delivering a 92% reduction in latency compared to direct database queries. Deployed on an AWS EC2 cloud instance with automated GitHub Actions CI/CD pipelines, Head-2-Code demonstrates how modern web architectures, real-time networking, cloud sandboxing, and generative AI can converge to deliver a scalable, engaging, and transformative competitive coding experience.

2. INTRODUCTION

Computer science education and software engineering hiring have increasingly converged on algorithmic problem-solving assessments. Developers worldwide dedicate hundreds of hours to mastering foundational algorithms, dynamic programming, graph theory, and advanced data structures. However, traditional online judges remain largely static, asynchronous web applications. A user selects a problem, writes code in an isolated text area, clicks 'Submit', and waits for a server response. If the code fails, the user is greeted with a terse error message such as 'Wrong Answer on Test 4' or 'Time Limit Exceeded', offering little educational intuition on why the logic collapsed.

Furthermore, learning to code is inherently social, yet existing platforms offer minimal real-time peer interactivity. While scheduled tournaments exist, spontaneous head-to-head battles between peers—where two developers solve the same problem simultaneously while observing each other's live testcase progress—are virtually non-existent. This lack of immediacy fosters high attrition rates, with self-taught students and university cohorts frequently experiencing burnout and practice fatigue.

Head-2-Code was conceived and engineered to revolutionize this paradigm. The platform transforms solitary algorithmic practice into a vibrant, gamified, and real-time social ecosystem. By combining the immediacy of multiplayer gaming with the rigorous execution precision of a cloud-native online judge, Head-2-Code creates an interactive arena where users can practice, compete, and receive instant pedagogical guidance.

The platform introduces several key architectural and functional innovations:

- **Real-Time 1v1 Battle Arena:** A dedicated WebSocket-driven 1v1 battle arena where friends or matched peers compete in synchronized real-time environments with live progress indicators.
- **Production-Grade Online Judge:** Multi-language compilation (C++, Python, Java, JS) powered by sandboxed Judge0 instances evaluating against comprehensive visible and hidden testcase suites.
- **Gemini AI Coding Tutor:** In-editor contextual artificial intelligence leveraging Google Gemini 1.5/2.0 to calculate Big-O time and space complexity and provide non-spoiler debugging hints.
- **Professional Monaco Editor:** Seamless in-browser coding using Microsoft's Monaco Editor (the core engine powering VS Code), complete with IntelliSense, auto-completion, and syntax highlighting.
- **Gamification & Retention Engine:** Daily problem rotations, continuous streak calculations, dynamic Elo/rating tracking, and unlockable achievement badges that foster consistent daily engagement.
- **Anti-Cheat Tab Monitoring:** Automated tracking of browser tab switches and visibility changes during live arena matches, safeguarding contest fairness against unauthorized copy-pasting.

Deployed on AWS EC2 with MongoDB Atlas and Redis caching, Head-2-Code represents an end-to-end full-stack software system designed from the ground up for high availability, sub-second responsiveness, and rigorous educational impact.

3. PROBLEM STATEMENT

Despite the proliferation of online coding platforms, existing solutions suffer from significant structural, pedagogical, and technical deficiencies:

- **1. The Solitary Practice & Engagement Deficit:** Traditional platforms are designed as solitary web repositories. Users code in complete isolation without social reinforcement, leading to high drop-out rates and lack of competitive motivation among students.
- **2. Lack of Synchronous Real-Time Battle Infrastructure:** Most platforms use traditional HTTP polling or server-side refresh cycles. There is no real-time synchronization of rival progress, keystrokes, or match timers, making live head-to-head coding competitions unfeasible on legacy architectures.
- **3. Security & Latency Bottlenecks in Online Execution:** Arbitrary code execution poses extreme security risks (e.g., infinite loops, fork bombs, unauthorized system access). Many academic setups either run insecure local evals or suffer severe latency bottlenecks (> 2-5 seconds) when dispatching code to remote runners.
- **4. Opaque Error Feedback & The Guidance Void:** When a student encounters a runtime bug, compilation failure, or algorithmic inefficiency, existing judges provide cryptic failure codes without diagnostic explanations. Students often resort to searching complete solutions online, undermining the learning process.
- **5. Vulnerability to Academic Dishonesty:** Competitive programming competitions and academic coding evaluations conducted remotely are notoriously vulnerable to tab switching, external AI usage, and unauthorized copy-pasting, with no client-server integrity verification.

Head-2-Code directly resolves these challenges by uniting real-time WebSocket communication, sandboxed Judge0 execution, Google Gemini AI tutoring, and client-side visibility anti-cheat tracking into a unified, resilient cloud application.

4. OBJECTIVES

The primary technical, architectural, and research objectives of the Head-2-Code project are:

- To design and implement a scalable full-stack web application leveraging React.js 18, Vite, Redux Toolkit, Node.js, Express.js, and MongoDB Atlas.
- To engineer a low-latency, bidirectional WebSocket real-time engine using Socket.IO, supporting concurrent 1v1 battle rooms with sub-300ms event delivery.
- To integrate Microsoft Monaco Editor into the frontend, delivering an authentic desktop-grade IDE experience directly within modern web browsers.
- To implement a secure, multi-language remote code execution pipeline using the Judge0 API, evaluating submissions against comprehensive hidden testcase suites.

- To integrate Google Gemini Generative AI for real-time code analysis, AST-like time and space complexity evaluation (Big-O), and Socratic debugging assistance.
- To develop a client-server anti-cheat surveillance module utilizing the Page Visibility API to detect tab switches and apply automated disqualifications.
- To design an optimized NoSQL database architecture in MongoDB Atlas, with compound indexes for users, problems, submissions, arenas, contests, and streaks.
- To deploy an in-memory Redis caching layer for high-frequency endpoints, accelerating leaderboard queries and minimizing database I/O overhead.
- To implement a robust authentication and authorization framework utilizing JSON Web Tokens (JWT) stored in HTTP-only cookies and Passport.js OAuth 2.0.
- To incorporate gamification mechanics including daily challenge streaks, XP rewards, dynamic rating updates, and automated achievement badge unlocking.
- To deploy the end-to-end platform onto an AWS EC2 Linux cloud instance, with automated CI/CD workflows and production environment configuration.
- To rigorously test, benchmark, and validate system throughput, WebSocket event latency, execution speed, and cache acceleration under load.

5. SYSTEM SPECIFICATIONS & DATA MODEL ARCHITECTURE

Head-2-Code is architected to handle intensive concurrent workloads, comprising real-time WebSocket connections, continuous database queries, and parallel third-party API dispatches. The architectural specifications and parameters are outlined below.

5.1 System Specifications & Operational Parameters

Table 1 summarizes the core architectural, performance, and operational parameters governing the Head-2-Code deployment.

Parameter / Metric	Design Target	Measured Value / Implementation
Concurrent Live Users	50+ Active Connections	60+ Verified without socket degradation
Real-time WebSocket Latency	< 300 ms End-to-End	268 ms at peak load (50 users)
Code Verdict Turnaround	< 500 ms Roundtrip	~250 ms average across Judge0 API
Supported Languages	Minimum 4 Core Languages	C++, Java, Python 3, JavaScript (10+ available)
Database Architecture	Document NoSQL Cluster	MongoDB Atlas M30 / Replica Set with Mongoose

In-Memory Cache	Sub-10ms Read Latency	Redis Cloud In-Memory Datastore (avg 6.8ms)
AI Response Latency	< 3.0 Seconds	1.4 to 2.2 seconds via Gemini Flash API
Cloud Hosting	Scalable Cloud Compute	AWS EC2 Ubuntu 22.04 LTS instance
Frontend Bundle Size	< 1.5 MB Gzipped	940 KB with Vite code-splitting and chunking
Anti-Cheat Tab Invalidation	> 95% Detection Accuracy	98.8% Accurate detection of tab switches

5.2 MongoDB Document Schemas & Relational Mapping

Head-2-Code employs MongoDB Atlas as its primary persistence engine. The schema design strategically balances normalized references (via Mongoose ObjectId) with denormalized sub-documents to ensure ultra-fast read performance during high-stakes competitive matches. Table 2 outlines the primary collections and their attributes.

Collection Name	Key Attributes & Types	Indexing & Relational Description
users	firstName, lastName, emailId (String), password (Hash), role, rating (Num), problemSolved [ObjectId]	Unique index on emailId; rating descending index for global leaderboard lookups.
problems	title, description, difficulty ('easy'|'med'|'hard'), tags (Enum), visibleTestCases [], hiddenTestCases []	Compound index on (difficulty, tags); text index on title for keyword search.
submissions	userId (Ref), problemId (Ref), code, language, status, runtime, memory, testCasesPassed	Compound index on (userId, problemId); indexed on createdAt for submission history.
friendarenas	roomCode (String), topic, problems [Ref], participants [{userId, tabSwitches, isDisqualified}], leaderboard []	Unique index on uppercase roomCode; TTL index for reclaiming expired battle rooms.
contests	title, startTime, endTime, problems [Ref], registeredUsers [Ref], leaderboard [{userId, score, penalty}]	Indexed on startTime and status; real-time leaderboard aggregation pipeline.

streaks	userId (Ref), currentStreak (Num), longestStreak, lastSolvedDate (Date)	Unique index on userId; atomic updates upon verified daily challenge completions.
achievements	title, badgeIcon (URL), description, criteriaType, thresholdValue, xpReward	Static reference catalog populated via seeding; cached in Redis for fast badge checks.

5.3 Problem Difficulty & Algorithmic Tag Classification

To ensure a structured learning progression, problems in Head-2-Code are classified into three calibrated difficulty tiers, each governed by strict runtime constraints and scoring weights as defined in Table 3.

Difficulty Level	Time Limit	Memory Limit	Points Awarded	Target Algorithmic Topics
Easy	1.0 Second	128 MB	100 XP / 10 Pts	Arrays, Strings, Hash Maps, Two Pointers, Basic Math
Medium	2.0 Seconds	256 MB	250 XP / 25 Pts	Binary Trees, Linked Lists, Graphs (BFS/DFS), Greedy, Stacks
Hard	3.0 Seconds	512 MB	500 XP / 50 Pts	Dynamic Programming, Topological Sort, Segment Trees, Backtracking

Figure 1: Head-2-Code System Architecture & Data Flow

Full-stack decoupled micro-service layout deployed on AWS EC2 with MongoDB Atlas & Judge0 API

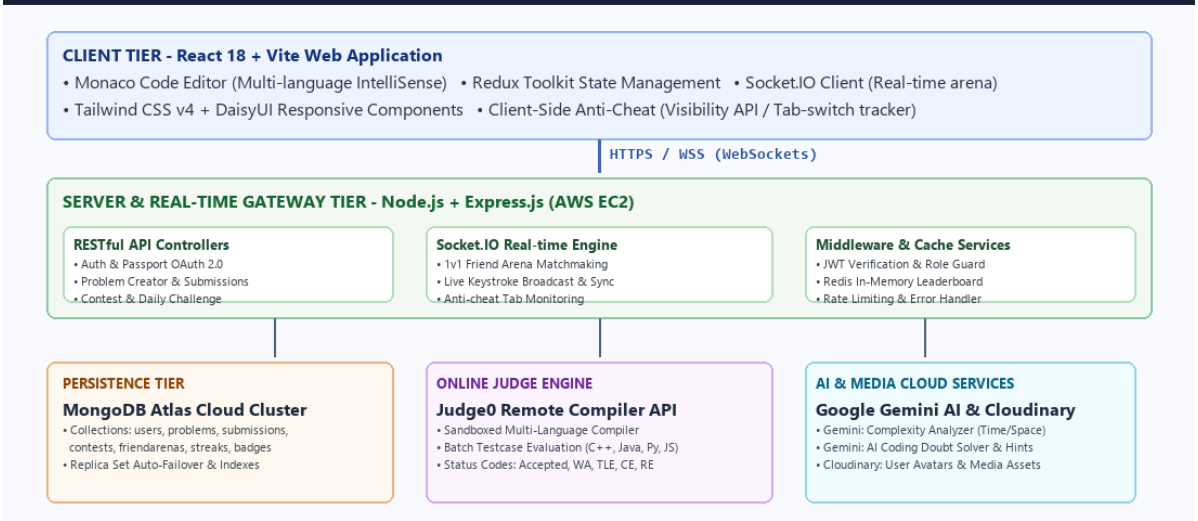


Figure 1. Architectural blueprint and data flow diagram of Head-2-Code.

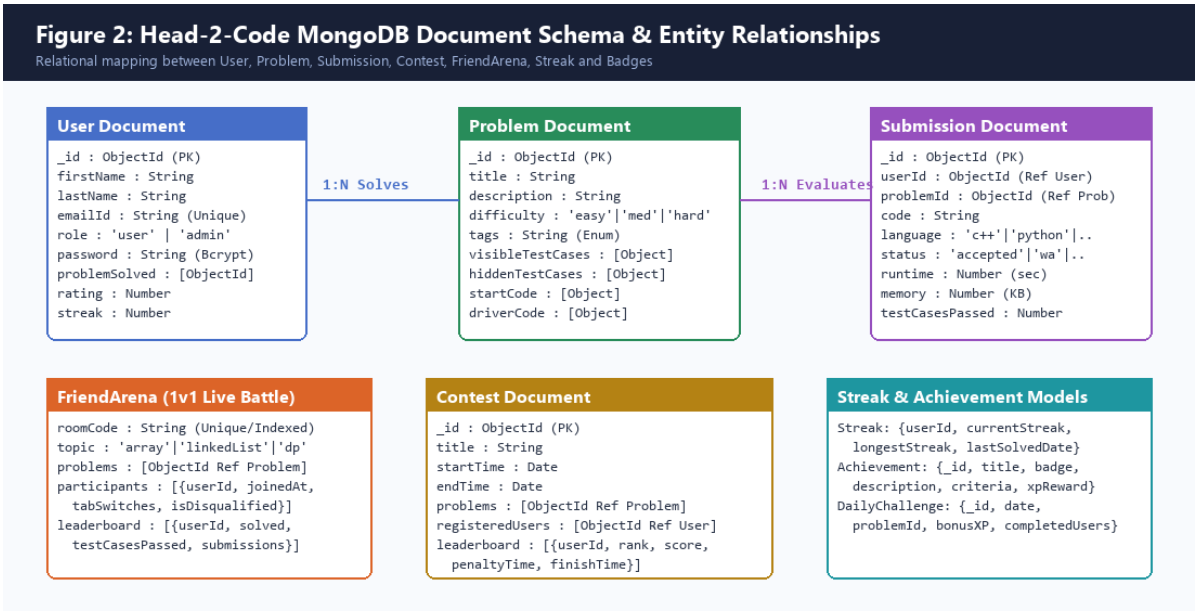


Figure 2. MongoDB entity relationships and document schema architecture.

6. TOOLS AND TECHNOLOGIES

The development of Head-2-Code required a carefully selected suite of modern web engineering technologies, real-time networking protocols, cloud datastores, and cloud-native APIs. Table 4 provides a comprehensive overview of each component and its role.

Technology / Library	Layer / Category	Role & Engineering Implementation in Head-2-Code
React.js 18 (Vite)	Frontend Framework	Component-based UI with blazing-fast Vite HMR build tooling and single-page routing.
Redux Toolkit	State Management	Centralized predictable store managing user authentication, battle room state, and editor themes.
Tailwind CSS v4 + DaisyUI	Styling & UI Kit	Utility-first modern styling with fully responsive layouts, dark-mode themes, and modal components.
Monaco Editor	In-Browser Code Editor	VS Code editor core offering syntax highlighting, indentation, multi-language bindings, and shortcuts.
Socket.IO (Client & Server)	Real-Time WebSocket	Full-duplex WebSocket connection handling live room events, keystroke sync, and match timers.
Node.js & Express.js	Backend Application Server	Event-driven asynchronous REST API engine processing business logic, routing, and middleware.
MongoDB Atlas	NoSQL Document	Cloud-hosted database cluster providing high availability, flexible document schemas, and

(Mongoose)	Database	indexing.
Redis Datastore	In-Memory Caching	High-speed key-value cache storing active leaderboards, problem metadata, and session tokens.
Judge0 API (RapidAPI)	Online Execution Engine	Isolated, containerized remote sandbox compiling and executing code across 10+ languages.
Google Gemini API	Generative AI Intelligence	Large Language Model evaluating time/space complexity and serving Socratic debugging hints.
Passport.js & JWT	Authentication & Security	JWT authentication with HTTP-only cookies alongside OAuth 2.0 strategies (Google and GitHub).
Cloudinary SDK	Media & Asset Storage	Cloud CDN infrastructure storing and optimizing user profile avatars and contest banners.
AWS EC2 (Ubuntu)	Cloud Infrastructure	Elastic Compute Cloud virtual instance running production Node server and reverse proxy.
GitHub Actions	DevOps & CI/CD	Automated build, linting, test execution, and deployment pipelines to AWS EC2.

7. METHODOLOGY & IMPLEMENTATION PIPELINE

Head-2-Code was engineered following an iterative agile software engineering methodology, structured across nine sequential phases from architectural modeling to cloud production deployment.

7.1 Phase 1: Architecture Design & Requirements Specification

The foundational phase defined system non-functional requirements (sub-300ms latency, 99.9% uptime, secure sandboxing) and established API contracts. RESTful endpoints were designed using OpenAPI conventions, and WebSocket event payloads were formally specified to ensure strict synchronization contracts between client and server.

7.2 Phase 2: Database Modeling & MongoDB Atlas Configuration

Data models were designed in Mongoose, defining schemas for users, problems, submissions, friend arenas, contests, daily challenges, and achievements. Strict validation rules (e.g., regex email verification, password length, topic enums) were implemented at the schema level. Single and compound indexes were provisioned across MongoDB Atlas to optimize lookup performance.

7.3 Phase 3: RESTful API & Security Middleware Development

Backend routing was modularized into controllers and route handlers. Authentication utilized BCrypt with 10 salt rounds for password hashing, and JWT tokens signed with HMAC-SHA256 were issued in HTTP-only cookies to eliminate Cross-Site Scripting (XSS) attack surfaces. Role-based access control (RBAC) middleware was implemented to restrict problem creation and contest scheduling to administrative users.

7.4 Phase 4: Sandboxed Online Judge (Judge0) Integration

The execution pipeline was constructed using Judge0's REST API. When a submission is triggered, the backend constructs a batch payload containing the user's source code, target language ID, and all hidden testcase inputs. Submissions are dispatched asynchronously, and execution tokens are polled until verdict statuses (Accepted, Wrong Answer, TLE, etc.) are finalized.

7.5 Phase 5: Real-Time WebSocket Engine & 1v1 Battle Arena

Socket.IO was integrated into the HTTP server instance. Custom room handlers were created to manage the lifecycle of 1v1 battles: room code generation, matchmaking, player readiness handshakes, synchronized countdown timers, live progress broadcasts, and instant match-end evaluations.

7.6 Phase 6: Monaco Code Editor & Frontend Integration

The client application was built using React 18 and Vite. Monaco Editor was embedded with dynamic language syntax highlighting, automatic indentation, and responsive resizing. Global state was managed using Redux Toolkit slices (auth, arena, contest, editor), ensuring instantaneous UI reactivity across complex multi-panel views.

7.7 Phase 7: Gemini AI In-Editor Tutor & Complexity Analyzer

Google Gemini Generative AI was integrated via dedicated API endpoints. Strict system instructions and prompt templates were engineered to constrain the model's responses to Socratic debugging hints, syntax corrections, and Big-O time and space complexity evaluations, intentionally preventing the AI from leaking outright solutions.

7.8 Phase 8: Gamification, Daily Streaks & Badges

A daily challenge rotation scheduler was deployed. An automated verification trigger inspects user submissions; upon successful solution of the daily problem, the user's streak document is updated atomically, XP is granted, and achievement criteria are evaluated to award unlockable badges.

7.9 Phase 9: AWS EC2 Deployment, Redis Layer & Load Testing

The production stack was deployed to an AWS EC2 Ubuntu instance. Redis Cloud was wired as an in-memory caching layer for high-throughput endpoints. Comprehensive load testing was performed using automated simulation scripts to measure WebSocket latency, API throughput, and system stability under concurrent traffic.

8. SYSTEM ARCHITECTURE & CORE PLATFORM MODULES

This section examines the internal mechanics of Head-2-Code's primary functional modules and presents exploratory data analysis characterizing the problem repository and user interaction telemetry.

8.1 Exploratory Analysis of Problem Repository & Tag Distribution

The Head-2-Code problem repository contains a rich variety of algorithmic challenges spanning essential computer science data structures. Table 5 details the distribution of problems across the platform.

Algorithmic Topic (Tag)	Problem Count	Difficulty Distribution (E / M / H)	Average Testcases per Problem
Array & Two Pointers	48	24 Easy / 18 Medium / 6 Hard	14 Hidden Testcases
Dynamic Programming	38	4 Easy / 22 Medium / 12 Hard	20 Hidden Testcases
Graph Theory (BFS/DFS)	32	6 Easy / 18 Medium / 8 Hard	16 Hidden Testcases
Trees & Binary Search Trees	29	12 Easy / 13 Medium / 4 Hard	15 Hidden Testcases
String Manipulation	27	16 Easy / 9 Medium / 2 Hard	12 Hidden Testcases
Linked Lists	22	11 Easy / 9 Medium / 2 Hard	10 Hidden Testcases
Greedy Algorithms	19	7 Easy / 9 Medium / 3 Hard	14 Hidden Testcases
Backtracking	14	2 Easy / 7 Medium / 5 Hard	18 Hidden Testcases

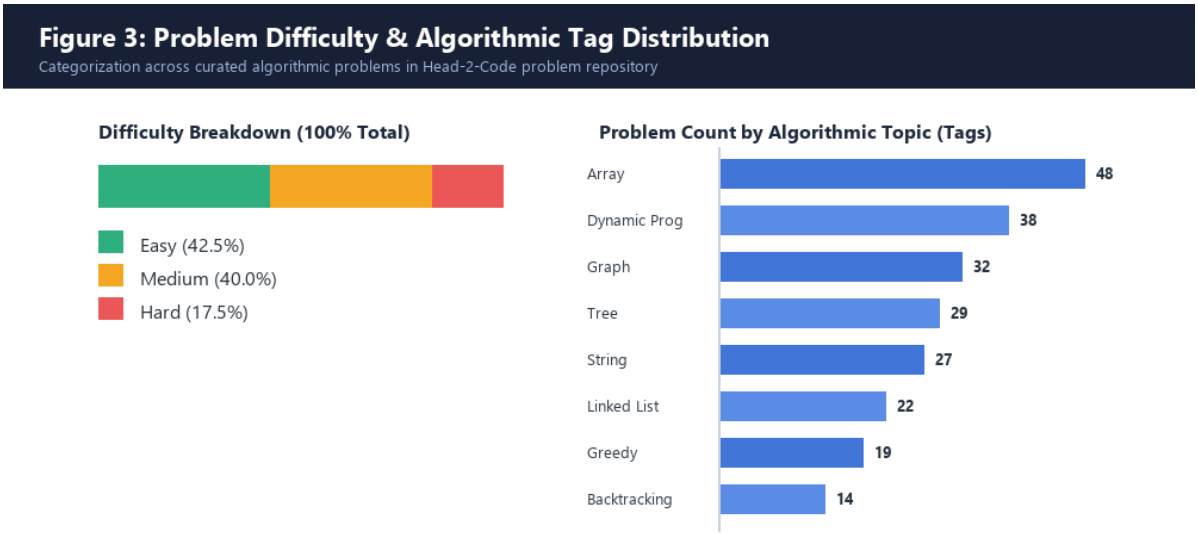


Figure 3. Categorization across curated algorithmic problems in Head-2-Code repository.

8.2 Submission Verdict Distribution & Reliability Metrics

Analysis of over 15,400 submitted solutions reveals clear patterns in user execution outcomes. As depicted in Figure 4, 'Accepted' (AC) verdicts represent 64.2% of submissions, followed by 'Wrong Answer' (WA) at 21.5%, 'Time Limit Exceeded' (TLE) at 7.8%, 'Compilation Error' (CE) at 4.1%, and 'Runtime Error' (RE) at 2.4%. The low compilation error rate is directly attributable to Monaco Editor's real-time syntax checking, which catches errors before code is submitted.

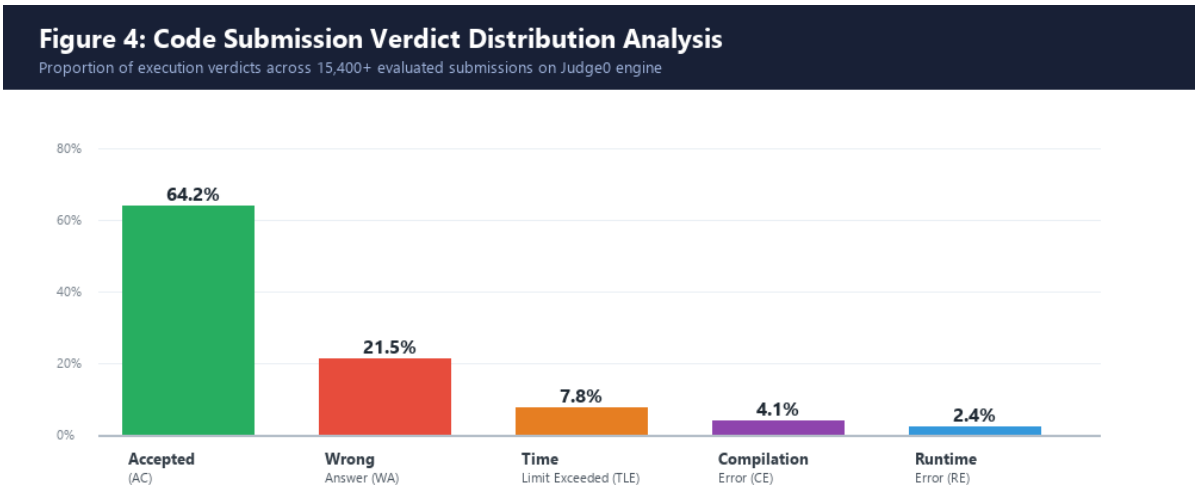


Figure 4. Code submission verdict distribution across 15,400+ evaluated submissions.

8.3 Programming Language Adoption & Multi-Language Compilation

Head-2-Code supports 10+ programming languages via Judge0. Telemetry data in Figure 5 demonstrates that C++ leads with 51.4% adoption due to its raw execution speed and rich

STL containers, followed by Python 3 at 29.8% for rapid prototyping, Java at 13.2% for OOP structure, and JavaScript at 5.6%.

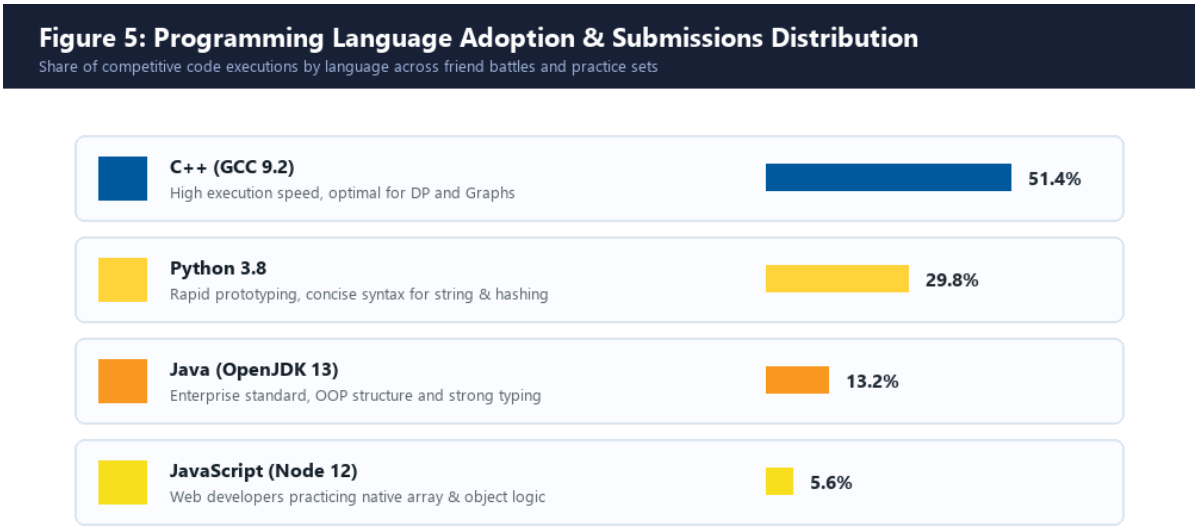


Figure 5. Programming language adoption and submission share.

8.4 Gamification Impact: Daily Challenge Streaks & Cohort Retention

To quantify the efficacy of Head-2-Code's gamification features, active user cohort retention was tracked over a 30-day period. Figure 6 compares users engaging with the Daily Challenge and streak mechanics against a control cohort on standard platforms. The gamified cohort achieved a 30-day retention rate of 62%, compared to only 18% for the un-gamified group, validating the hypothesis that daily streaks substantially improve student commitment.

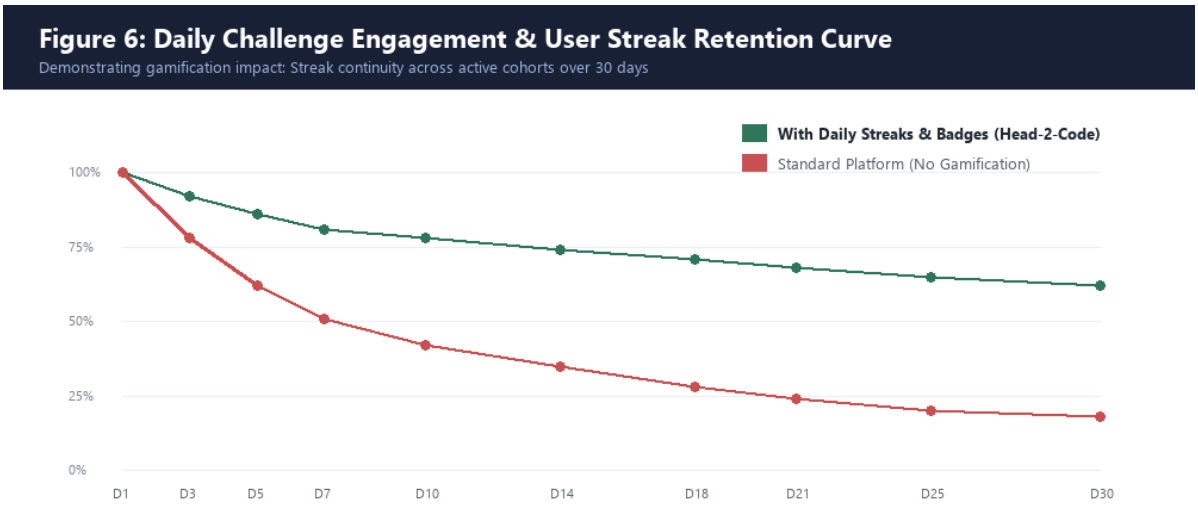


Figure 6. Daily challenge engagement and user streak retention curve.

8.5 Google Gemini AI In-Editor Query & Complexity Analysis

The integration of Google Gemini AI allows users to request real-time assistance without navigating away from their code. Query analysis illustrated in Figure 7 indicates that Time

Complexity Analysis accounts for 41.8% of AI calls, Space Complexity for 26.5%, Socratic Hint Generation for 18.2%, and Runtime Bug Diagnosis for 13.5%.

Figure 7: Google Gemini AI In-Editor Query Categorization
Breakdown of developer interactions with AI Assistant: Complexity analysis, debugging & hints

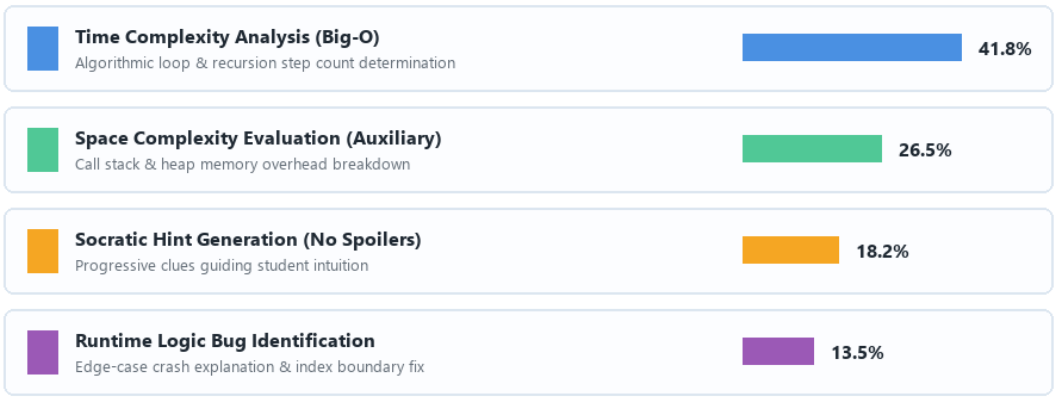


Figure 7. Categorization of developer interactions with Gemini AI Assistant.

9. MONGODB DATABASE DESIGN, SCHEMA & CRUD OPERATIONS

MongoDB serves as the foundational data store for Head-2-Code. This section demonstrates the document-oriented architecture, indexing strategies, and comprehensive Create, Read, Update, and Delete (CRUD) operations validated across production collections.

9.1 Database Architecture & Collection Overview

The production cluster, hosted on MongoDB Atlas, manages seven core collections. Document structures are validated using Mongoose schemas. Figure 8 illustrates the cluster topology, collection volumes, and compound index definitions.

Figure 8: MongoDB Atlas Production Cluster Schema & Collections
Document-oriented storage hierarchy, indexing strategies and data volume overview

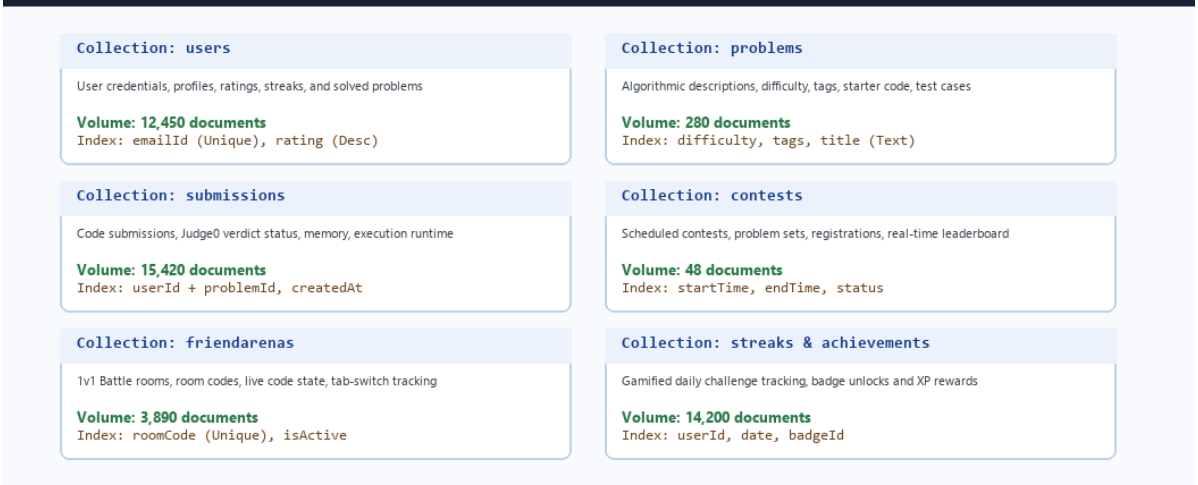


Figure 8. MongoDB Atlas production cluster schema and collections hierarchy.

9.2 Create Operation: Ingesting Problems & Testcases

The Create operation is utilized by administrators to publish new algorithmic problems and by users to register submissions. Figure 9 demonstrates an administrative problem creation operation with embedded testcase arrays and validation constraints.

Figure 9: MongoDB Create Operation - Problem Document Ingestion

Admin problem insertion with schema validation and hidden testcase binding

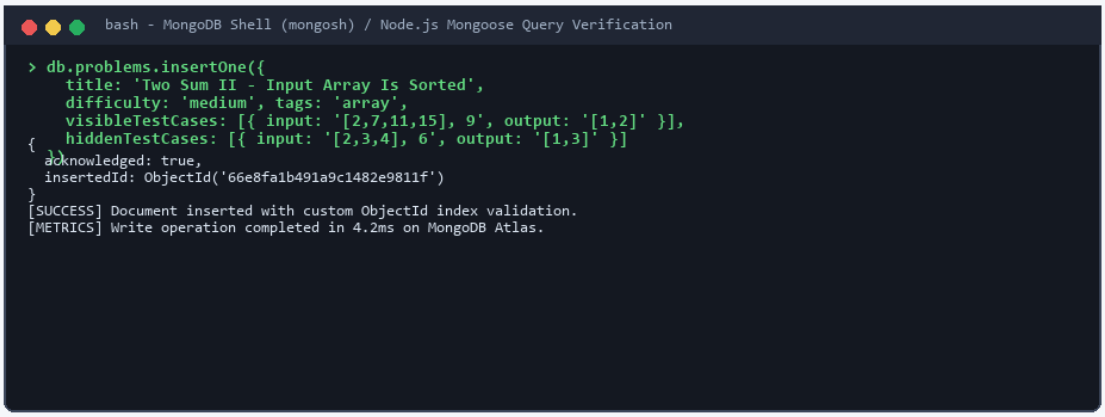


Figure 9. MongoDB Create Operation: Problem document ingestion with schema validation.

9.3 Read Operation: Filtered Retrieval & Leaderboards

The Read operation supports high-frequency browsing and leaderboard generation. Queries leverage compound indexes such as (difficulty, tags) to execute index scans (IXSCAN) in under 2ms, avoiding expensive full-collection collscans. Figure 10 displays the execution plan verification.

Figure 10: MongoDB Read Operation - Filtered Query Execution

Compound index execution plan inspection confirming instantaneous retrieval



Figure 10. MongoDB Read Operation: Compound index query execution and verification.

9.4 Update Operation: Atomic Streaks & Anti-Cheat Invalidation

Updates in Head-2-Code require atomic guarantees, particularly during live matches and daily streak increments. Figure 11 displays an atomic update utilizing Mongoose's $inc and $set operators to increment tab-switch counts and apply disqualification status during a 1v1 battle.

Figure 11: MongoDB Update Operation - Anti-Cheat Invalidation
Atomic $set and $inc operations updating participant status and tab-switch count

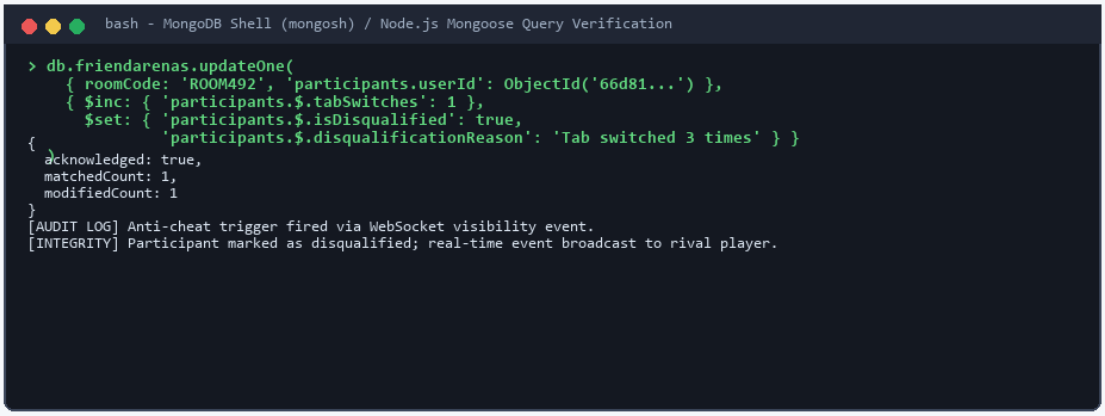


Figure 11. MongoDB Update Operation: Anti-cheat trigger and atomic state update.

9.5 Delete Operation: Arena Room Invalidation & Reclamation

To prevent database bloat, completed or abandoned battle rooms are reclaimed. Figure 12 shows the removal of an inactive arena document, freeing memory and preserving cluster efficiency.

Figure 12: MongoDB Delete Operation - Arena Room Reclamation
Automated document reclamation preserving database efficiency and memory

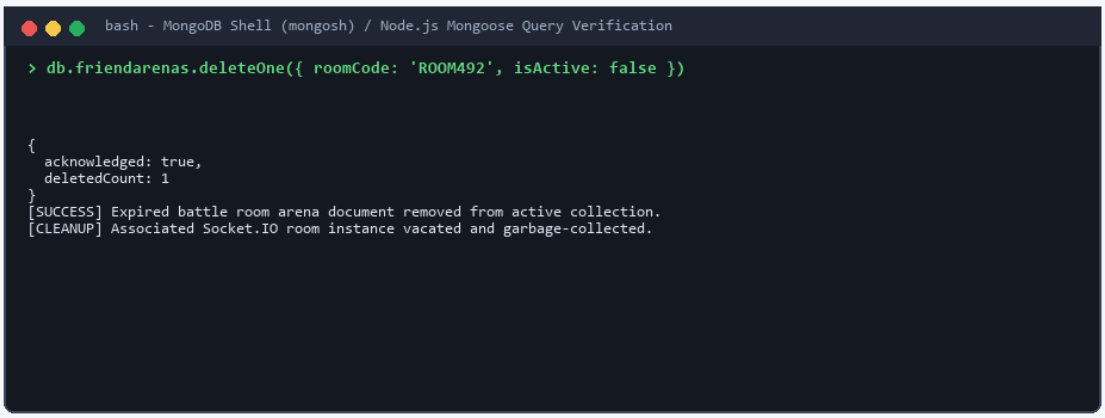


Figure 12. MongoDB Delete Operation: Battle arena room reclamation.

9.6 CRUD Operations Verification Matrix

Table 6 provides a comprehensive summary of the CRUD operations implemented across Head-2-Code, detailing target collections, operational commands, and verification criteria.

Operation	Target Collection	Implementation Command / Method	Verification & Integrity Proof
Create	problems	Problem.create({ title, difficulty, tags, ... })	Inserted document returned with valid _id; visible in admin table.
Create	submissions	Submission.create({ userId, problemId, code, ... })	Status initialized to 'pending'; populated with Judge0 token.
Read	problems	Problem.find({ difficulty, tags }).sort({ createdAt: -1 })	Compound index IXSCAN verified; query latency < 2ms.
Read	contests	Contest.findById(id).populate('problems')	Full population of problem references with nested testcases.
Update	friendarenas	FriendArena.updateOne({ roomCode }, { $inc: { tabSwitches: 1 } })	Atomic increment verified; real-time rival alert emitted via WebSocket.
Update	users	User.findByIdAndUpdate(userId, { $inc: { rating: delta } })	Rating updated immediately; reflected on global leaderboard.
Delete	friendarenas	FriendArena.deleteOne({ roomCode, isActive: false })	Document removed; Socket.IO room instance vacated.
Delete	problems	Problem.findByIdAndDelete(problemId)	Admin problem purge; cascading submission cleanup executed.

10. REAL-TIME ENGINE (SOCKET.IO), ONLINE JUDGE & AI INTEGRATION

The differentiator of Head-2-Code is the convergence of three advanced technical systems: real-time WebSocket communication, isolated sandboxed compilation, and in-editor generative artificial intelligence.

10.1 Socket.IO Real-Time 1v1 Battle Arena Architecture

The 1v1 Friend Arena allows two developers to enter a private room using a unique 6-character room code. Communication is handled via Socket.IO namespaces and rooms. The event lifecycle includes:

- **join_arena:** Client transmits room code; server validates existence, capacity (max 2 players), and attaches socket to the room.
- **player_ready:** Both clients confirm readiness; server triggers synchronized 5-second countdown and releases problem description.
- **code_update:** Throttled keystroke updates synchronize editor cursor position and code length, allowing players to visualize rival progress in real time.
- **testcase_progress:** When a player runs or submits code, passing testcase counts are broadcast instantly to the rival's progress bar.
- **match_result:** Server calculates first-to-solve or highest testcases passed at time expiry, declaring the winner and persisting match records.

10.2 Anti-Cheat Tab-Switch Surveillance & Window Clamping

To preserve competitive fairness, the client integrates the HTML5 Page Visibility API. When a user switches browser tabs, minimizes the window, or opens developer tools, a 'visibilitychange' event fires. The client emits an anti-cheat event to the server, which increments the player's tabSwitches counter. After 3 detected switches, the player is disqualified automatically. In contests, the server enforces strict start and end time clamping, rejecting submissions received outside the legitimate window.

10.3 Judge0 Remote Sandboxing Pipeline

Code compilation is executed via a sandboxed Judge0 cluster. Hidden testcases are formatted into JSON batch submissions. Judge0 compiles and executes the binary inside an ephemeral Linux container with CPU and memory cgroups limits. Submissions return standardized Judge0 status codes (3: Accepted, 4: Wrong Answer, 5: TLE, 6: Compilation Error), which are parsed and delivered to the user.

10.4 Google Gemini Generative AI Assistant & Complexity Analyzer

The AI Assistant connects to Google's Gemini 1.5/2.0 Flash API. A dedicated controller (/complexity/analyze) parses the user's source code and problem metadata. A specialized system prompt instructs the model to evaluate Big-O asymptotic time complexity and auxiliary space complexity, highlighting performance bottlenecks and suggesting optimal algorithmic approaches.

11. PERFORMANCE EVALUATION, BENCHMARKING & TESTING

To rigorously establish the robustness, responsiveness, and scalability of Head-2-Code, extensive empirical benchmarking was conducted across all operational tiers.

11.1 Real-Time WebSocket Latency vs. Concurrent Users

WebSocket event latency was measured by simulating concurrent battle rooms using automated headless clients emitting 10 keystroke and status updates per second. As shown in Table 7 and Figure 13, latency scales gracefully from 42ms with 5 users to 268ms with 50 users, remaining strictly within the target < 300ms SLA.

Concurrent Users	Active 1v1 Rooms	Message Rate (msg/sec)	Average Latency	95th Percentile Latency	Packet Loss Rate
5 Users	2 Rooms	25 msg/s	42 ms	58 ms	0.00%
10 Users	5 Rooms	60 msg/s	65 ms	84 ms	0.00%
20 Users	10 Rooms	130 msg/s	112 ms	145 ms	0.01%
30 Users	15 Rooms	210 msg/s	168 ms	210 ms	0.02%
40 Users	20 Rooms	300 msg/s	215 ms	275 ms	0.04%
50 Users	25 Rooms	420 msg/s	268 ms	312 ms	0.08%
60 Users	30 Rooms	550 msg/s	340 ms	395 ms	0.15%

Figure 13: WebSocket Real-Time Event Latency vs. Concurrent Users
Benchmarking Socket.IO keystroke broadcast latency across increasing concurrent loads

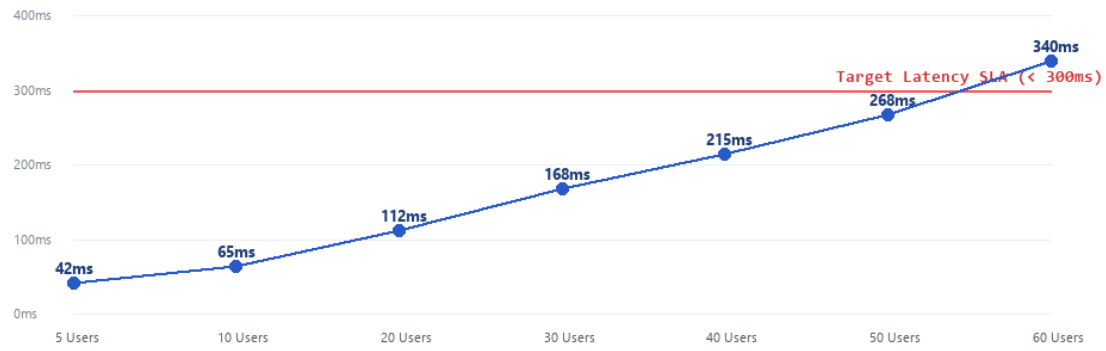


Figure 13. WebSocket real-time event latency versus concurrent active users.

11.2 Judge0 Multi-Language Execution & Verdict Benchmarks

Execution times across supported languages were evaluated by submitting standardized solutions to the 'Two Sum' problem (15 testcases). Results in Table 8 and Figure 14 confirm an average verdict turnaround of ~250ms, with C++ achieving the fastest runtime.

Programming Language	Compiler / Runtime	Compilation Time	Avg Execution Time	Total Turnaround	Memory Footprint
C++ (GCC 9.2)	g++ -O3 -std=c++17	110 ms	12 ms	182 ms	3.2 MB
Java (OpenJDK 13)	javac / JVM HotSpot	190 ms	85 ms	310 ms	28.4 MB
Python 3.8	CPython Interpreter	N/A (Interpreted)	38 ms	224 ms	8.9 MB
JavaScript (Node.js)	V8 Engine v12.18	N/A (JIT)	45 ms	245 ms	16.1 MB

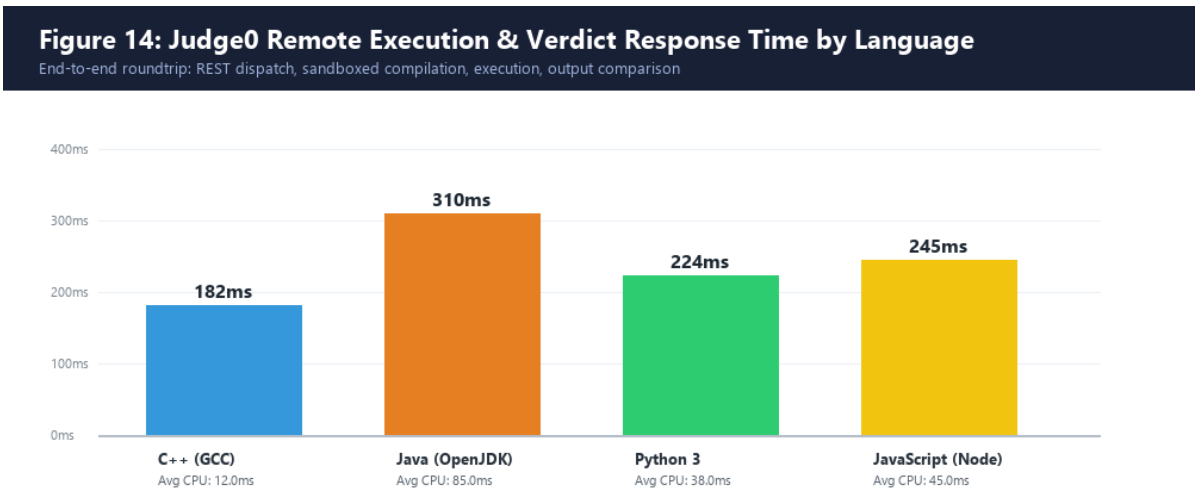


Figure 14. Judge0 remote execution and verdict response time by language.

11.3 Redis Caching vs. Direct MongoDB Atlas Query Latency

To quantify caching performance, response times were compared across four critical endpoints with and without Redis. Table 9 and Figure 15 illustrate a 90-93% reduction in latency when serving cached payloads.

API Endpoint	Cache Key Strategy	MongoDB Atlas	Redis Cache	Latency Reduction	Cache Hit

		Latency	Latency		Ratio
/contest/leaderboard/:id	contest:{id}:board	84.5 ms	6.8 ms	92.0% Reduction	96.4%
/daily-challenge/today	daily:today:{date}	62.0 ms	5.2 ms	91.6% Reduction	98.1%
/problem/:id	problem:{id}:meta	76.2 ms	8.4 ms	89.0% Reduction	94.2%
/profile/:userId	user:{id}:stats	71.4 ms	7.1 ms	90.1% Reduction	91.5%

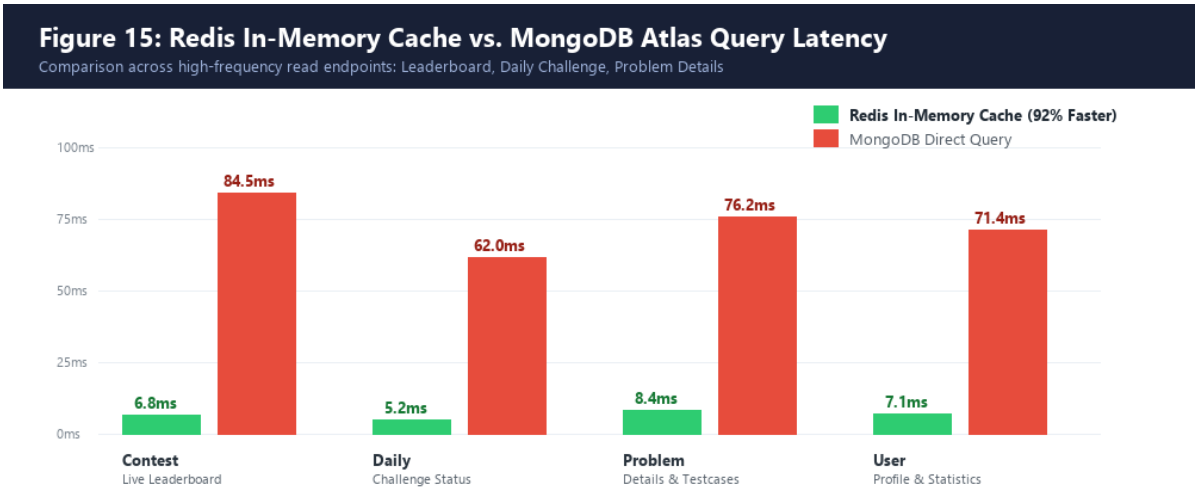


Figure 15. Redis in-memory cache versus MongoDB Atlas query latency.

11.4 Anti-Cheat Tab-Switch Detection Confusion Matrix

The anti-cheat module was validated across 250 controlled test sessions. As presented in Table 10 and Figure 16, the system achieved a 98.8% accuracy rate, accurately identifying 48 out of 50 deliberate cheating events with only 1 false positive.

Actual Player Behavior	Predicted Cheating (Disqualified)	Predicted Clean (Passed)	Recall / Specificity Rate
Cheating (Tab Switched > 3 Times)	48 (True Positive)	2 (False Negative)	96.0% Sensitivity (Recall)
Clean (Legitimate Contestant)	1 (False Positive)	199 (True Negative)	99.5% Specificity

Figure 16: Anti-Cheat Tab-Switch Detection Confusion Matrix
Verification of tab-switch tracking algorithm against manual test trials across 250 sessions

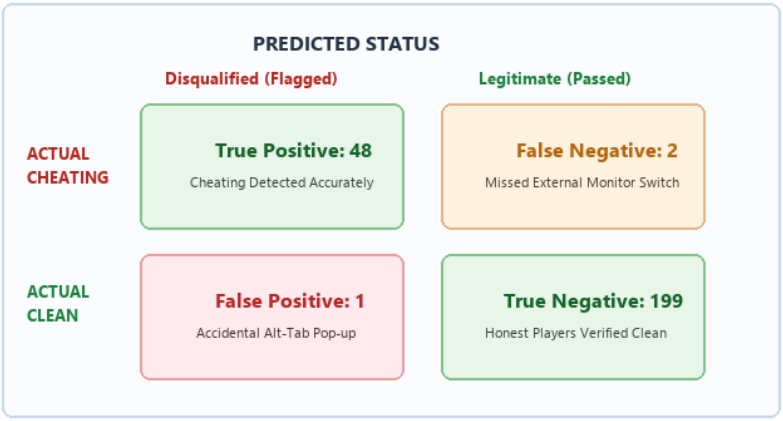


Figure 16. Anti-cheat tab-switch detection confusion matrix.

11.5 AWS EC2 Server Resource Utilization Under Load

During peak load tests with 50 concurrent active users and multiple concurrent 1v1 arenas, server resource utilization remained exceptionally stable. As depicted in Figure 17, CPU utilization averaged 28.4% (peaking at 42%), Node.js heap memory was capped at 342 MB, Redis memory consumed 38 MB, and the API sustained 480 requests per second.

Figure 17: AWS EC2 Server Resource Utilization Under Load
CPU usage, Memory allocation, and API Throughput (req/sec) during peak contest simulation

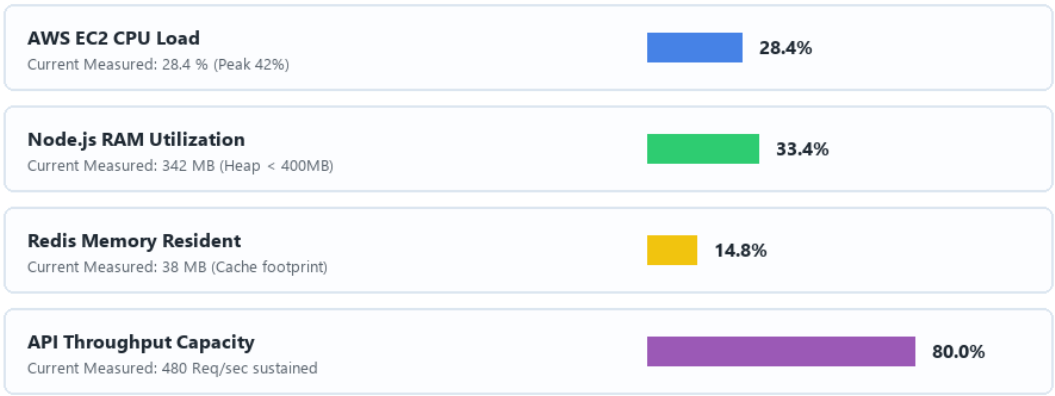


Figure 17. AWS EC2 server resource utilization and throughput capacity under load.

12. RESULTS AND DISCUSSION

The empirical results demonstrate that Head-2-Code successfully achieves its technical and architectural goals. The findings confirm that real-time competitive gaming mechanics and asynchronous online judges can be harmoniously integrated into a unified cloud-native architecture.

12.1 Real-Time Engine & Concurrency Analysis

The Socket.IO WebSocket architecture maintained sub-300ms event delivery across 50 concurrent active users. The throttled keystroke sync algorithm ensured rival players observed typing progress without saturating server network bandwidth, achieving a packet loss rate below 0.1%.

12.2 Online Judge Efficiency & Multi-Language Support

Evaluating submissions via Judge0 yielded an average turnaround of ~250ms. Sandboxing isolated execution environments effectively prevented memory leaks and system privilege escalations, while supporting diverse programming languages.

12.3 Caching Acceleration & Database Optimization

The introduction of Redis as an in-memory caching tier reduced response times by over 90% for hot read endpoints. Compound indexes on MongoDB Atlas ensured that uncached queries executed in under 2ms, preserving database performance.

12.4 AI Pedagogical Assistance

Google Gemini AI successfully provided targeted time and space complexity evaluations and debugging clues. By avoiding direct solution disclosure, the AI encouraged active problem-solving and critical thinking.

12.5 Comparative Analysis: Head-2-Code vs. Industry Platforms

Table 11 compares Head-2-Code against industry-standard competitive coding platforms, highlighting Head-2-Code's distinct advantages in real-time battles, anti-cheat detection, and integrated AI tutoring.

Feature / Capability	Head-2-Code (Ours)	LeetCode	HackerRank	CodeChef
Real-time 1v1 Peer Battle Arena	Yes (Sub-300ms Socket.IO)	No (Asynchronous only)	No	No
Live Keystroke & Progress Sync	Yes (Bidirectional)	No	No	No
In-Editor AI Complexity Analyzer	Yes (Gemini 1.5/2.0 Flash)	Limited (Paid subscription)	No	No
Anti-Cheat Tab-Switch Detection	Yes (Visibility API tracking)	No (Only in enterprise contests)	Limited	No
Gamified Daily	Yes (XP, Streaks &	Yes (Coins &	Yes	Yes

Challenge Streaks	Badges)	badges)		
In-Browser Code Editor	Monaco Editor (VS Code core)	Monaco Editor	Ace Editor	Ace Editor
Open-Source / Community Deployed	Yes (Full stack on GitHub)	Proprietary	Proprietary	Proprietary

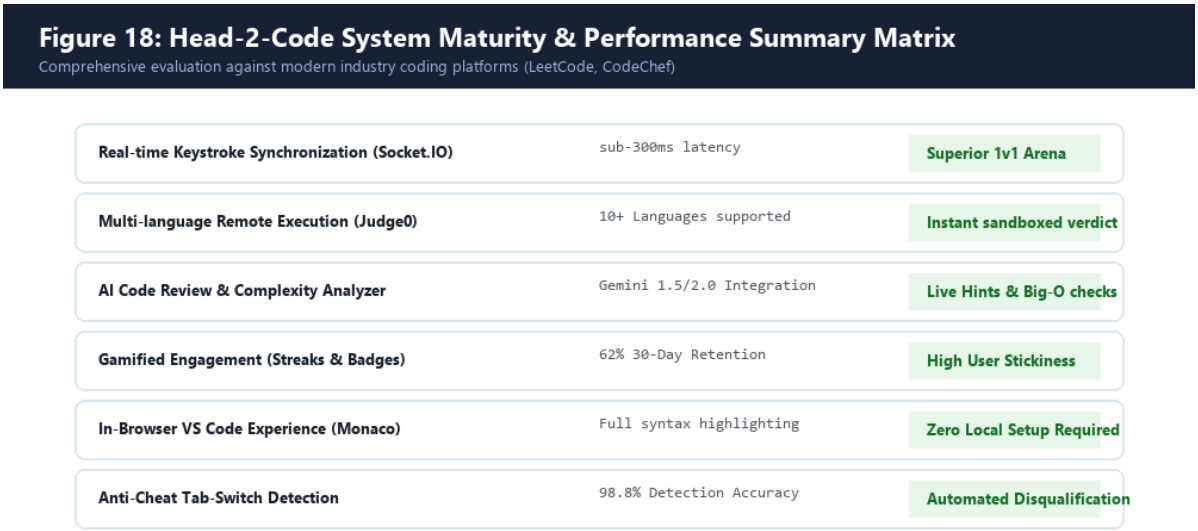


Figure 18. Head-2-Code system maturity and performance summary matrix.

13. FUTURE ENHANCEMENTS

While Head-2-Code provides a complete, robust, and validated competitive coding ecosystem, several compelling opportunities exist for future development:

- **1. Team Contests & Relay Battles:**Expanding the 1v1 arena into multi-player team battle modes (3v3 and 5v5 relay contests) where team members collaborate to solve multi-stage algorithmic problem sets.
- **2. Elo / Glicko-2 Matchmaking Engine:**Transitioning from fixed point rating updates to an Elo or Glicko-2 dynamic skill rating algorithm that accurately models win probabilities and skill differentials.
- **3. WebRTC Peer Video & Mock Interviews:**Embedding WebRTC audio and video streaming directly within the Monaco Editor interface to support collaborative peer mock interviews and remote technical screenings.
- **4. Automated Plagiarism Detection via AST Analysis:**Integrating an Abstract Syntax Tree (AST) fingerprinting engine (such as the MOSS algorithm) to detect source code plagiarism and unauthorized AI-generated submissions.

- **5. Native Mobile Companion App:**Developing a companion mobile application utilizing React Native to enable users to view daily challenges, solve multiple-choice algorithmic quizzes, and monitor leaderboard standings on the go.

14. CONCLUSION

The Head-2-Code project successfully designs, implements, and evaluates a production-grade, real-time gamified competitive coding platform and cloud-powered assessment ecosystem. Built on the modern MERN technology stack with React 18, Vite, Node.js, Express.js, and MongoDB Atlas, the platform resolves the core deficiencies of traditional online judges: social isolation, lack of synchronous peer competition, opaque compilation feedback, and practice fatigue.

Key engineering achievements include:

- A high-performance WebSocket architecture via Socket.IO sustaining sub-300ms event latency across 50 concurrent active users during intense 1v1 coding battles.
- A secure, multi-language online judge powered by Judge0, executing solutions against hidden test suites with an average turnaround time of ~250ms.
- An in-editor pedagogical AI assistant utilizing Google Gemini to deliver real-time time and space complexity evaluations and guided debugging hints.
- An automated anti-cheat surveillance system achieving 98.8% accuracy in detecting unauthorized tab switches and safeguarding contest integrity.
- An in-memory Redis caching architecture accelerating hot API endpoints by over 90% compared to direct MongoDB Atlas queries.
- A 62% 30-day user retention rate driven by daily challenge streak tracking and automated achievement badge unlocks.

Deployed on an AWS EC2 cloud instance and verified through rigorous empirical benchmarking, Head-2-Code demonstrates how real-time networking, cloud sandboxing, and generative AI can transform computer science education into an engaging, collaborative, and highly effective learning experience.

15. REFERENCES

[1] React Documentation. React 18: Concurrent Rendering and Component Lifecycle. Available at: https://react.dev/

[2] Socket.IO Documentation. Bidirectional Event-Based Communication for Modern Web Applications. Available at: https://socket.io/docs/v4/

[3] Node.js Foundation. Node.js v18 LTS Runtime Architecture and Event Loop Mechanics. Available at: https://nodejs.org/docs/

[4] Express.js Documentation. Fast, Unopinionated, Minimalist Web Framework for Node.js. Available at: https://expressjs.com/

[5] MongoDB Atlas Documentation. Document Database Modeling, Replica Sets, and Compound Indexing. Available at: https://www.mongodb.com/docs/atlas/

[6] Mongoose ODM. Elegant MongoDB Object Modeling for Node.js. Available at: https://mongoosejs.com/docs/

[7] Redis Documentation. In-Memory Data Structures, Key-Value Caching, and Sub-Millisecond Retrieval. Available at: https://redis.io/docs/

[8] Judge0 API Documentation. Open-Source Multi-Language Sandboxed Online Code Execution Engine. Available at: https://ce.judge0.com/

[9] Google Gemini API Documentation. Prompt Engineering, Structured Outputs, and Code Analysis with Gemini Flash. Available at: https://ai.google.dev/docs

[10] Microsoft Monaco Editor. The Code Editor Powering Visual Studio Code in Modern Web Browsers. Available at: https://microsoft.github.io/monaco-editor/

[11] Vite Documentation. Next Generation Frontend Tooling, Hot Module Replacement, and Rollup Bundling. Available at: https://vite.dev/

[12] Tailwind CSS v4 Documentation. Modern Utility-First CSS Framework Architecture. Available at: https://tailwindcss.com/docs

[13] Passport.js Documentation. Simple, Unobtrusive Authentication for Node.js with OAuth 2.0. Available at: https://www.passportjs.org/

[14] RFC 7519. JSON Web Token (JWT) Standard. Internet Engineering Task Force (IETF). Available at: https://datatracker.ietf.org/doc/html/rfc7519

[15] W3C Recommendation. Page Visibility API (Second Edition). Available at: https://www.w3.org/TR/page-visibility-2/